CODE OUTPUT REPORT

Customer Purchase Prediction Using Logistic Regression

Output from the Python mini-project script

Note: The program uses 240 synthetic demonstration records. The output below is reproducible with the supplied script and random seed.

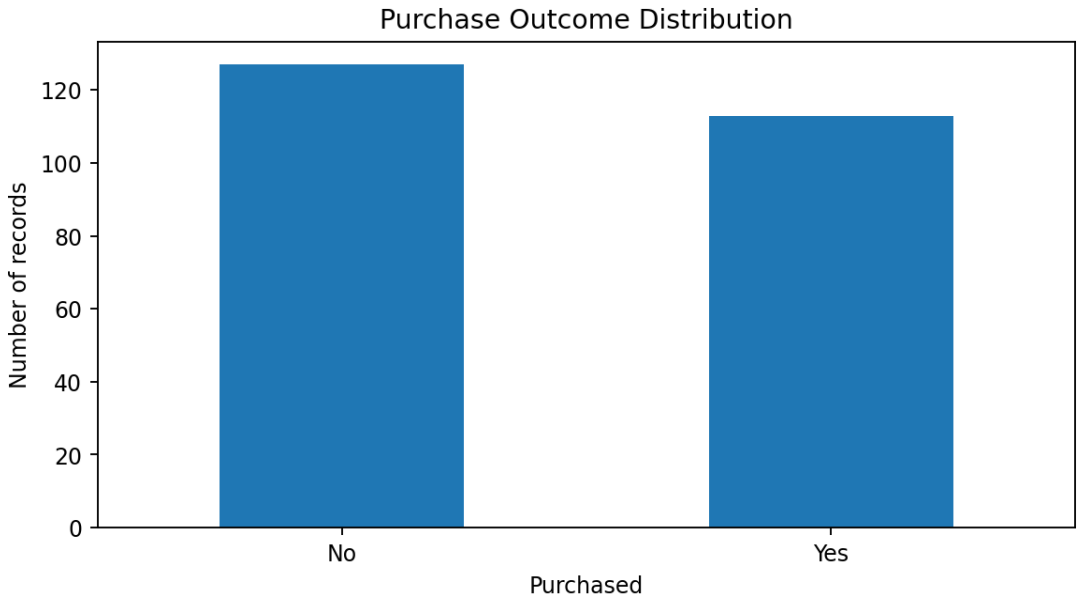
1. First 10 Dataset Rows

```
Age_group Income Student Purchased
   Middle   High      No       Yes
   Middle   High      No       Yes
   Senior    Low     Yes       Yes
   Middle Medium      No       Yes
    Young Medium     Yes        No
   Senior   High     Yes       Yes
   Middle Medium      No        No
   Senior Medium     Yes       Yes
    Young Medium     Yes       Yes
   Middle Medium      No       Yes
```

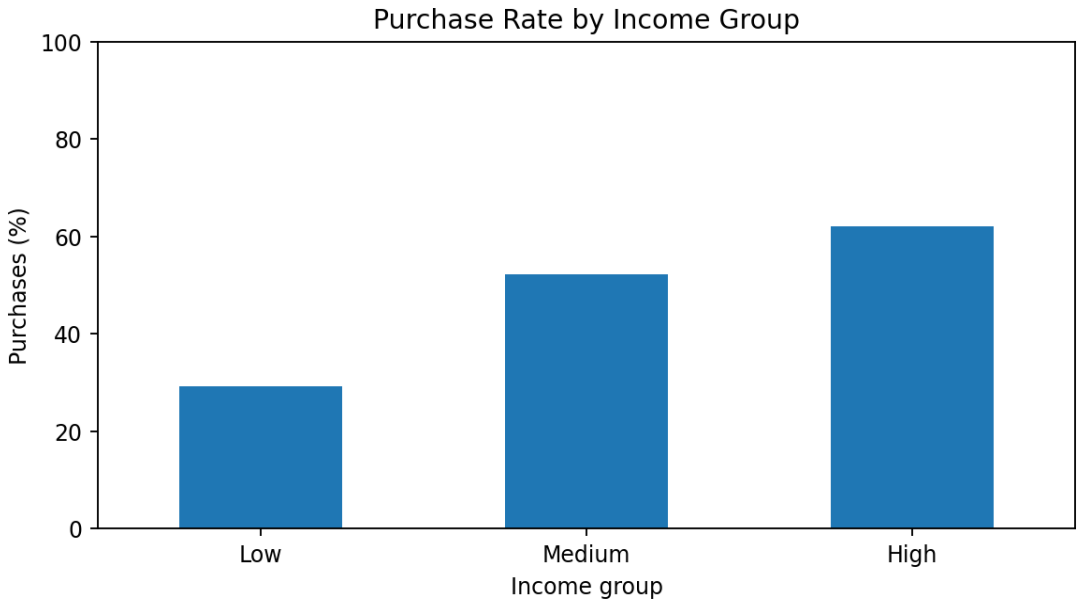
2. Purchase Counts

```
Purchased
No     127
Yes    113
```

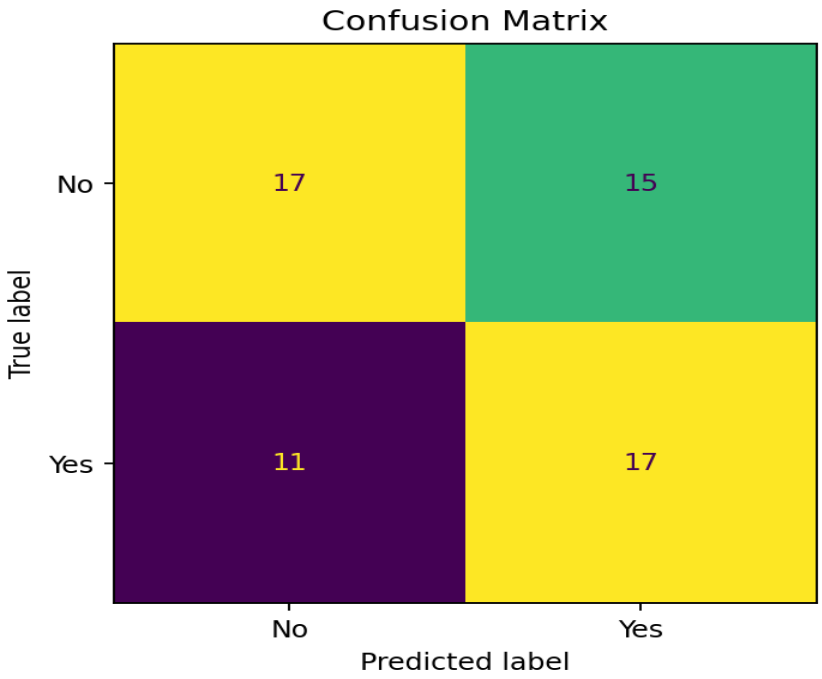
3. Graph 1 — Purchase Outcome Distribution



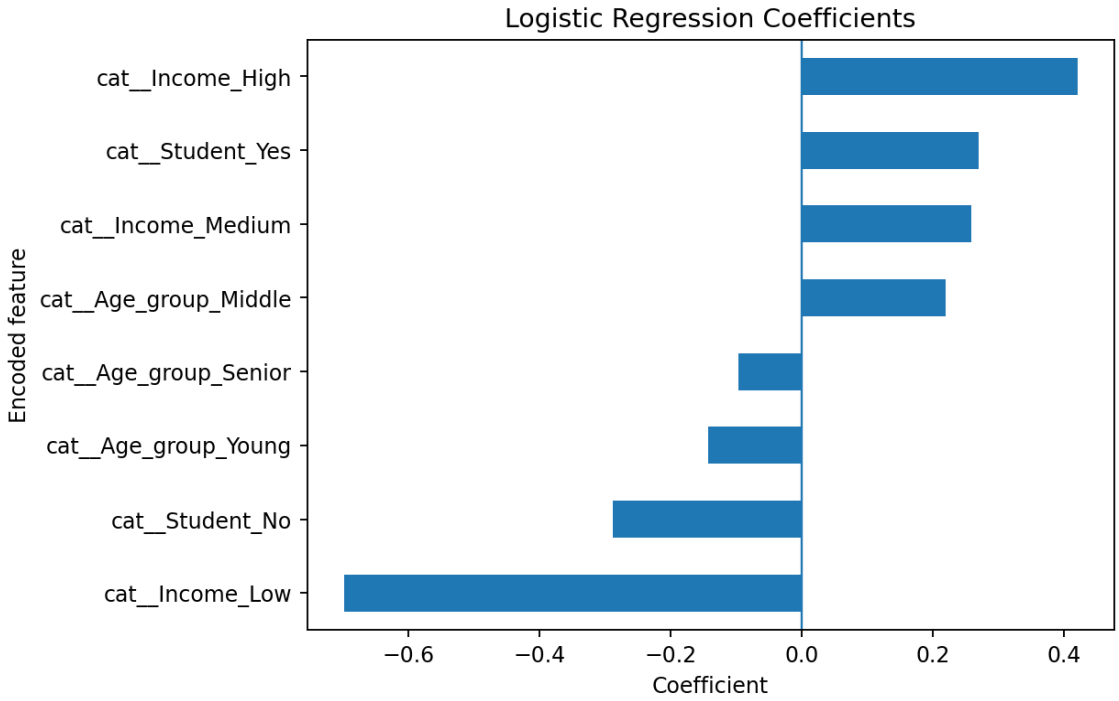
4. Graph 2 — Purchase Rate by Income Group



5. Graph 3 — Confusion Matrix



6. Graph 4 — Logistic Regression Coefficients



7. Model Evaluation Output

```
Accuracy: 0.567

Classification report:
              precision    recall  f1-score   support

          No       0.61      0.53      0.57        32
         Yes       0.53      0.61      0.57        28

    accuracy                           0.57        60
   macro avg       0.57      0.57      0.57        60
weighted avg       0.57      0.57      0.57        60
```

Confusion matrix (actual rows and predicted columns; class order No, Yes)

Actual / Predicted	No	Yes
No	17	15
Yes	11	17

8. Example Prediction

```
Input customer:
Age_group Income Student
    Young Medium     Yes
Predicted purchase outcome: Yes
Estimated purchase probability: 58.8%
```

Interpretation

The model classified the example customer as **Yes** and estimated a 58.8% probability for the Yes class. These are model outputs on simulated data only and should not be interpreted as real-world customer predictions.

Important note

The graphs and metrics are generated from synthetic data for an educational demonstration. The exact values are tied to the script and fixed random seed.

MINI PROJECT REPORT

Customer Purchase Prediction Using Logistic Regression

Python • Pandas • NumPy • Matplotlib • scikit-learn

Objective

Build a machine-learning classifier to predict whether a customer is likely to purchase a product using age group, income group, and student status.

Dataset note

This report uses a synthetic dataset generated for demonstration. It is not real customer data, and its metrics are not evidence of real-world buying behaviour.

Prepared as an educational Python mini project

1. Abstract

This project demonstrates a complete classification workflow: generate sample data, visualize patterns, encode categorical variables, train a Logistic Regression model, evaluate it, and predict an example customer's outcome.

2. Problem Statement

Businesses may want to understand patterns associated with customer purchases. This project uses Age_group, Income, and Student as input features and Purchased (Yes/No) as the target label.

3. Objectives

• Create and inspect a structured dataset.
• Create charts with Matplotlib.
• Encode categorical values with OneHotEncoder.
• Train a Logistic Regression classifier.
• Evaluate it with accuracy, classification report, and confusion matrix.
• Predict an example customer's outcome.

4. Requirements and Execution

```
pip install pandas numpy matplotlib scikit-learn
python customer_purchase_prediction.py
```

5. Dataset Description

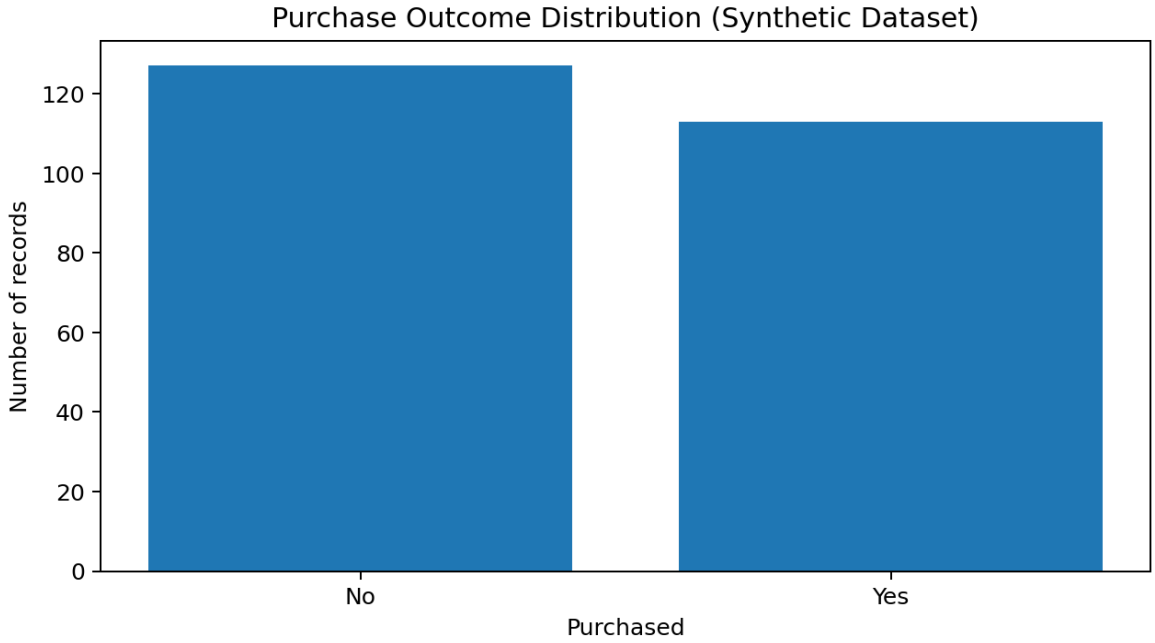
Column	Meaning	Example values
Age_group	Customer age category	Young, Middle, Senior
Income	Income category	Low, Medium, High
Student	Student status	Yes, No
Purchased	Target label	Yes, No

The 240 example records are generated with a fixed random seed. The labels are simulated for teaching and do not represent real customer behaviour.

6. Methodology

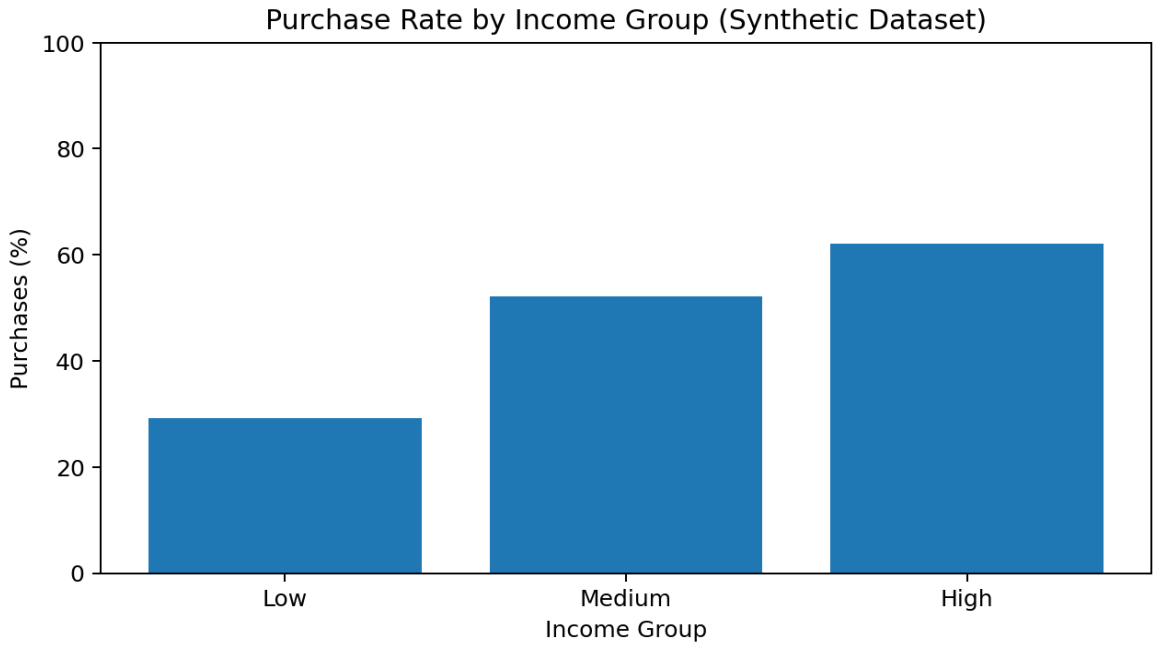
1) Generate data. 2) Explore the purchase outcomes. 3) Split records into 75% training and 25% testing. 4) Encode categorical inputs. 5) Train Logistic Regression. 6) Evaluate predictions. 7) Predict a new example customer.

7. Graph 1 — Purchase Outcome Distribution



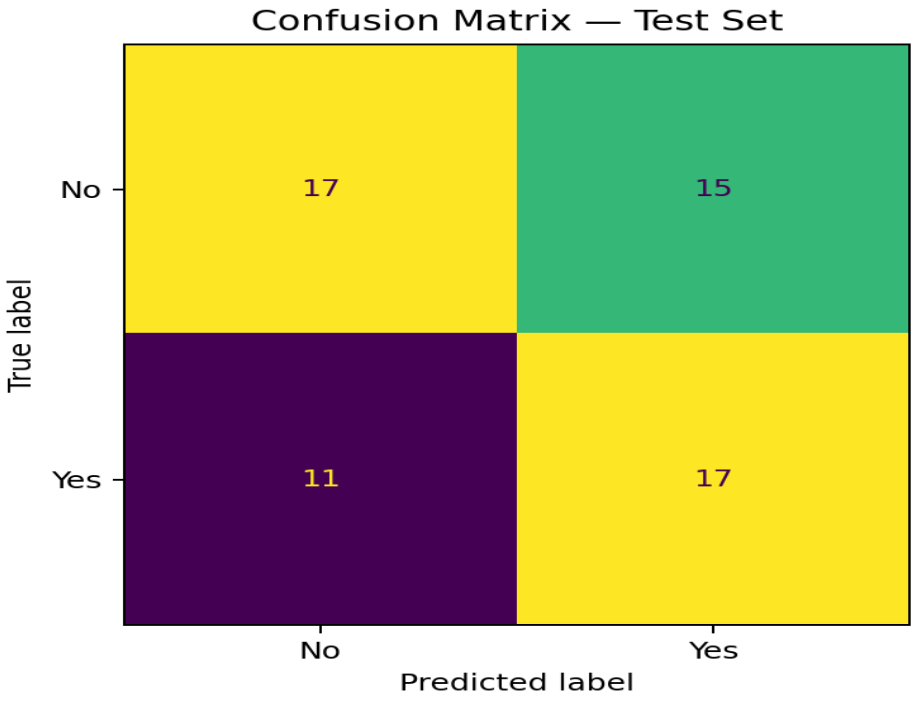
Shows the number of synthetic records labelled Yes and No.

Graph 2 — Purchase Rate by Income Group



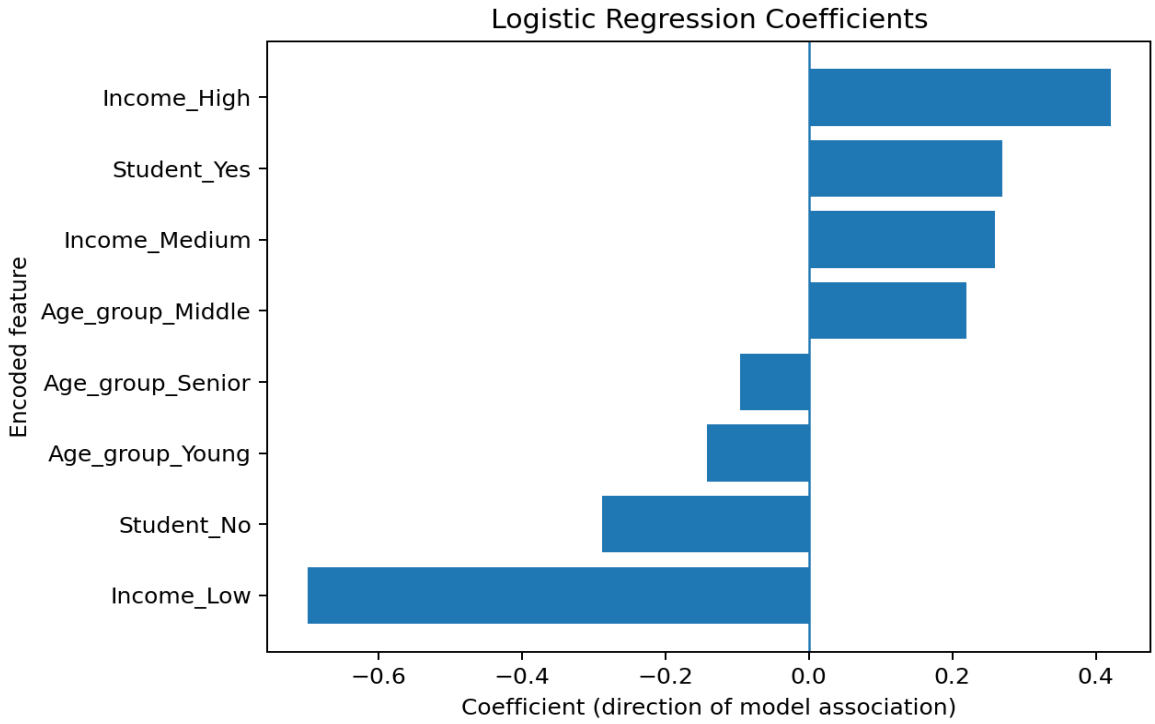
Shows the percentage labelled Purchased = Yes in each income group. This is only a pattern in the simulated data.

Graph 3 — Confusion Matrix



Rows represent actual labels and columns represent predicted labels, ordered No then Yes.

Graph 4 — Logistic Regression Coefficients



Coefficients show model associations after encoding. They do not prove causation.

8. Model Evaluation

Test-set accuracy for this synthetic run: **0.567** (56.7%).

Accuracy is the proportion of test predictions that are correct. Precision, recall, and F1-score are also printed by the Python script. Since the data is simulated, these values only demonstrate evaluation methods.

Confusion matrix values (actual rows, predicted columns)

Actual / Predicted	No	Yes
No	17	15
Yes	11	17

9. Example Prediction

The script predicts a label and estimated purchase probability for a sample customer: Young, Medium income, Student = Yes. The output is printed when the program runs.

10. Limitations

- Synthetic data does not represent a real market.
- Only three input features are used.
- A small test set can produce unstable metrics.
- Model associations do not establish causation.
- Practical use requires representative, lawfully collected data and further validation.

11. Conclusion

This project demonstrates an end-to-end supervised learning workflow for a Yes/No classification task, including data preparation, visualization, categorical encoding, Logistic Regression, evaluation, and prediction.

12. Viva Questions

1. What is classification? Predicting a discrete category.
2. What is Logistic Regression? A supervised algorithm often used for classification.

3. Why use OneHotEncoder? To convert categories into numeric indicator features.
4. Why split the data? To test on records not used for training.
5. What is accuracy? The proportion of correct predictions.
6. What is a confusion matrix? A comparison of actual and predicted labels.
7. Why use Matplotlib? To create data visualizations.
8. Is this real customer data? No, it is synthetic.

Appendix — Complete Python Code

The runnable script is also provided separately as customer_purchase_prediction.py.

```
"""
MINI PROJECT: Customer Purchase Prediction Using Logistic Regression
Dataset: Synthetic demo data generated within this script (not real customer records).
Requirements:
    pip install pandas numpy matplotlib scikit-learn
Run:
    python customer_purchase_prediction.py
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay
)

# 1. Create a reproducible sample dataset.
# Replace this section with a real, consented dataset for real-world use.
rng = np.random.default_rng(42)
n = 240

age_groups = rng.choice(
    ["Young", "Middle", "Senior"], size=n, p=[0.42, 0.36, 0.22]
)
incomes = rng.choice(
    ["Low", "Medium", "High"], size=n, p=[0.32, 0.42, 0.26]
)
students = np.where(
    age_groups == "Young",
    rng.choice(["Yes", "No"], n, p=[0.68, 0.32]),
    rng.choice(["Yes", "No"], n, p=[0.12, 0.88])
)
# Simulate purchase outcomes for a teaching demonstration.
# These rules create example data; they are not evidence about real customers.
logit = (
    -0.55
    + np.select(
        [age_groups == "Young", age_groups == "Middle", age_groups == "Senior"],
        [0.35, 0.25, -0.25]
    )
    + np.select(
        [incomes == "Low", incomes == "Medium", incomes == "High"],
        [-0.55, 0.15, 0.65]
    )
    + np.where(students == "Yes", 0.40, 0.0)
)
probability = 1 / (1 + np.exp(-logit))
purchased = np.where(rng.random(n) < probability, "Yes", "No")

df = pd.DataFrame({
    "Age_group": age_groups,
    "Income": incomes,
    "Student": students,
    "Purchased": purchased
})

print("\\n--- First 10 rows ---")
print(df.head(10))
print("\\n--- Purchase counts ---")
print(df["Purchased"].value_counts())

# 2. Explore data and create graph 1: purchase outcome distribution.
plt.figure(figsize=(7, 4))
df["Purchased"].value_counts().reindex(["No", "Yes"]).plot(kind="bar")
plt.title("Purchase Outcome Distribution")
plt.xlabel("Purchased")
plt.ylabel("Number of records")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 3. Graph 2: purchase rate by income group.
purchase_rates = df.groupby("Income")["Purchased"].apply(
    lambda values: (values == "Yes").mean() * 100
).reindex(["Low", "Medium", "High"])

plt.figure(figsize=(7, 4))
purchase_rates.plot(kind="bar")
plt.title("Purchase Rate by Income Group")
plt.xlabel("Income group")
plt.ylabel("Purchases (%)")
plt.ylim(0, 100)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# 4. Separate predictors (X) and target (y).
X = df[["Age_group", "Income", "Student"]]
y = df["Purchased"]

# 5. Split into training and testing sets.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 6. Convert categories into numeric features, then train Logistic Regression.
categorical_features = ["Age_group", "Income", "Student"]
preprocessor = ColumnTransformer(
```

```
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])
model_pipeline.fit(X_train, y_train)

# 7. Predict test records and evaluate the model.
y_pred = model_pipeline.predict(X_test)
print("\\n--- Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print("\\nClassification report:")
print(classification_report(y_test, y_pred, zero_division=0))

# 8. Graph 3: confusion matrix.
cm = confusion_matrix(y_test, y_pred, labels=["No", "Yes"])
ConfusionMatrixDisplay(
    confusion_matrix=cm, display_labels=["No", "Yes"]
).plot(colorbar=False)
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()

# 9. Graph 4: model coefficients (association, not causal effects).
classifier = model_pipeline.named_steps["classifier"]
feature_names = model_pipeline.named_steps["preprocessor"].get_feature_names_out()
coefficients = pd.Series(classifier.coef_[0], index=feature_names).sort_values()

plt.figure(figsize=(8, 5))
coefficients.plot(kind="barh")
plt.axvline(0, linewidth=1)
plt.title("Logistic Regression Coefficients")
plt.xlabel("Coefficient")
plt.ylabel("Encoded feature")
plt.tight_layout()
plt.show()

# 10. Predict purchase probability for a new example customer.
new_customer = pd.DataFrame([{
    "Age_group": "Young",
    "Income": "Medium",
    "Student": "Yes"
}])
predicted_class = model_pipeline.predict(new_customer)[0]
yes_index = list(model_pipeline.named_steps["classifier"].classes_).index("Yes")
purchase_probability = model_pipeline.predict_proba(new_customer)[0][yes_index]

print("\\n--- Example Prediction ---")
print(new_customer.to_string(index=False))
print(f"Predicted purchase outcome: {predicted_class}")
print(f"Estimated purchase probability: {purchase_probability:.1%}")

print("\\nNote: This is a synthetic educational demonstration. Results do not represent real customers.")
```